

Feature Engineer — Easy Guide

Crash course before the presentation · Retail Demand Forecasting (Walmart M5)

In one sentence

I turn raw sales rows into **38 numbers** that describe a product's situation, so the model has something meaningful to learn from.

What a "feature" is

A **feature** is one input column the model reads. The **label** (or target) is what we want to predict — units sold.

Raw data says: `"FOODS_3_090, store CA_3, 2016-03-05, sold 4"`. That alone predicts nothing about tomorrow.

Features add the context: what it normally sells, when it last sold, what day it is, what the price is, which department it's in.

RAW ROW -> FEATURE BUILDER -> 38 numbers -> LightGBM

The seven groups (32) + 6 shape features = 38

Group	How many	Examples
A Calendar	9	wday, month, year, is_weekend, events, snap
B Historical demand	8	lag_1, lag_7, lag_14, lag_28, rolling_mean_7/28, rolling_std_7/28
C Recency	3	days_since_last_sale, zero_streak_length, days_since_first_sale
D Listing	2	days_since_first_listing, pre_listing
E Price	4	sell_price, recent_avg_price, price_rel_to_recent_avg, price_is_missing
F Hierarchy	5	item_id, dept_id, cat_id, store_id, state_id
G Horizon	1	horizon (1–28)
Shape/cycle	6	wday_ratio_52w, wday_ratio_13w, snap_lift, weekend_lift, month_ratio, dom_ratio

The four you must be able to explain

Lag — the value N days ago. `lag_7` = what it sold exactly a week ago. Weekly rhythm: Saturday looks like last Saturday.

Rolling mean — the average of the last N days. `rolling_mean_28` is basically "how big is this product".

Rolling std (standard deviation) — how much the numbers bounce around. 3,3,3,3 → std 0 (steady). 0,0,12,0 → std large (spiky).

days_since_last_sale — how long since this product sold anything. This one exists **because 68% of our data is zero**. For a product that sells once a month, that beats any average.

The price feature people ask about

`price_rel_to_recent_avg` = today's price ÷ its average over the last **8 weeks**.

Why the ratio and not the price? "£2" means nothing on its own. "30% below its usual price" means a lot — it's a promotion signal.

Important: the model uses price as **context**, not as a cause. We measured what happens if you simulate a price cut and the response was non-monotone and sometimes backwards. That's why the app has **no price what-if slider**.

The six shape features, simply

They capture how *this specific product* behaves, not products in general.

`month_ratio` = this product's average sales in this month ÷ its overall average.

- 1.0 → this month is typical for it
- 1.4 → it sells 40% more than usual this month

Computed over a **728-day (2-year) window ending at the forecast origin**, and **shrunk toward 1.0** for low-volume products so three random sales don't create fake seasonality.

The rule that matters most: no leakage

Leakage = accidentally letting the model see information from *after* the moment it's predicting from. It makes a model look brilliant in testing and fail in reality.

Every feature is computed **at or before the forecast origin**.

Allowed anyway: calendar facts about the future — date, month, events, SNAP days, the weekly price. Those are genuinely published in advance. **Never allowed:** future sales.

How we prove it: overwrite all sales after the origin with 9999, rebuild every feature, and require them to come back **bit-for-bit identical**. We also confirm the *target* does change, so the test can't pass for a silly reason. Foundation suite: **46/46 passed**.

Two models, two feature sets

	Direct	Recursive
Features	38	32

The recursive model drops six: `days_since_last_sale`, `zero_streak_length`, `days_since_first_sale`, `days_since_first_listing`, `pre_listing`, `horizon`.

Why? The recursive model feeds its own predictions back in. A prediction of 0.3 would count as "a sale happened" and would corrupt `days_since_last_sale` a bit more on every one of the 28 steps. And `horizon` is pointless when the model always predicts exactly one day ahead.

Honest limitations we admit

- `zero_streak_length` largely duplicates `days_since_last_sale`.
- `pre_listing` duplicates `price_is_missing`.

We measured both, documented them, and kept them — because the champion was frozen on the measured configuration.

Feature importance rankings are not verified in our materials. Don't quote numbers for "the most important feature".

Files to name

- `03_RESEARCH/pipeline/features_v5.py` — the champion builder, `CHAMPION_FEATURES` (38)

- 03_RESEARCH/pipeline/champion_blend.py — REC_COLS_V5 (32)
- 03_RESEARCH/pipeline/config.py — PRICE_AVG_WINDOW_WEEKS = 8, HORIZON = 28

How to explain it to a judge

"My part is feature engineering. The raw data just says a product sold 4 units on a date — that predicts nothing on its own. So I build 38 features that describe the product's situation.

The core ones are lags and rolling statistics — what it sold 1, 7, 14 and 28 days ago, plus its 7- and 28-day average and variability. Then calendar features: weekday, month, events, SNAP days. Then price, where the useful signal is price relative to its own 8-week average, which is really a promotion indicator.

Because 68% of our data is zero, we have a group about recency — days since this product last sold anything. For rare-selling products that matters far more than an average.

And we added six per-series shape features: how this specific product behaves on this weekday, in this month, on SNAP days — computed over two years up to the forecast origin and shrunk toward neutral for low-volume items.

The strict rule is no leakage. Everything is computed at or before the forecast origin, and we prove it by overwriting future sales with 9999 and checking every feature comes back identical."

A worked example — one product, one moment

Say we are standing at the forecast origin and this product's last 28 days of sales looked like this:

```
... 0 0 3 0 0 0 2 0 0 0 0 4 0 0 1 0 0 0 0 0 0 2 0 0 0 0 0 0
                                     ^ origin
```

Some of the features that get built from it:

Feature	Value here	What it tells the model
lag_1	0	yesterday: nothing
lag_7	2	a week ago it sold 2
rolling_mean_28	≈ 0.43	it averages under half a unit a day
rolling_std_28	large relative to the mean	very spiky, not steady
days_since_last_sale	6	it *is* still alive, just quiet
zero_streak_length	6	same story from the other direction
snap	0 or 1	is the food benefit spendable that day
horizon	1...28	which of the 28 future days this row is for

Notice how little a single lag says here, and how much rolling_mean_28 and days_since_last_sale say. **That is the whole reason group C exists.**

Feature vs label — don't mix them up

	Feature	Label
What it is	Input the model reads	The answer we want
Example	lag_7 = 2	units sold tomorrow
Known at prediction time?	Yes, always	No — that's the point
How many	38	1

Training vs inference — where features are used

TRAINING	history	->	features + known answers	->	model learns
INFERENCE	history	->	features (no answers)	->	model predicts

The **same builder** produces the features both times. That is deliberate: if training and prediction built features differently, the model would be reading a different world than the one it learned from.

Quick answers

What's a feature? An input column the model reads. *How many?* 38 direct, 32 recursive. *What's a lag?* The value N days ago — we use 1, 7, 14, 28. *Why so many zero-related features?* Because 68% of the data is zero. *Is using next month's date leakage?* No — calendar facts are published in advance. Only future sales would be leakage. *Why do the two models differ?* The recursive one feeds its own predictions back, so recency features would get corrupted during the 28-step rollout.

30-second revision

38 features · 7 groups (calendar, demand, recency, listing, price, hierarchy, horizon) + 6 shape · lags 1/7/14/28 · rolling mean & std 7/28 · price vs 8-week average · recency group exists because of 68% zeros · shape features over a 728-day window, shrunk · recursive member uses 32, dropping recency + horizon · no leakage, proved by the 9999 corruption test · 46/46 foundation checks.